

SOFTWARE ENGINEERING COURSE

LECTURE 8

Software Evolution

Dr. Micheal Tuape



MODULE 1 SCHEDULE

Week	Dates	Topic	SE Chapters
Week 9	Oct. 28–Nov1, 2024	Software Testing & Quality Management	Chapter 8 & 24
Week 10	Nov. 4– Nov 8, 2024	Software Evolution & Configuration Management	Chapter 9 & 25
Week 11	Nov. 11– Nov 15, 2024	Software Project Management & Software Project Planning	None
Week 12	Nov. 18– Nov 22, 2024	Software Quality and Configuration	Chapter 22 & 23
Week 13	Nov. 25– Nov 29, 2024	Guest Lecture: Customer Success & Software Development	None
Week 14	December 2–6, 2023	Trends in Software Engineering	None





LEARNING OBJECTIVES

- »» Evolution processes
- »» Legacy systems
- »» Software maintenance





SOFTWARE CHANGE

»» Software change is inevitable

- New requirements emerge when the software is used;
- The business environment changes;
- Errors must be repaired;
- New computers and equipment is added to the system;
- The performance or reliability of the system may have to be improved.

»» A key problem for all organizations is implementing and managing change to their existing software systems.



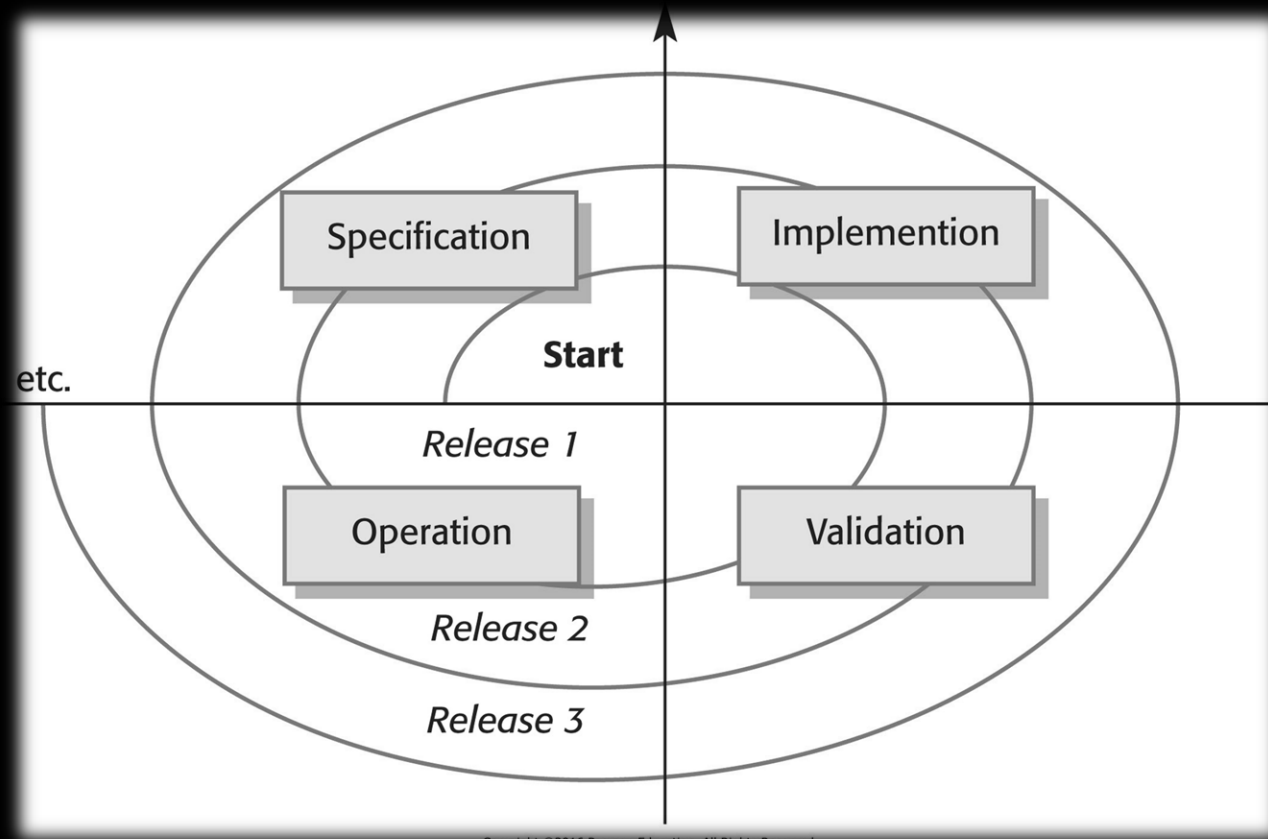


IMPORTANCE OF EVOLUTION

- »» Organisations have huge investments in their software systems – they are critical business assets.
- »» To maintain the value of these assets to the business, they must be changed and updated.
- »» The majority of the software budget in large companies is devoted to changing and evolving existing software rather than developing new software.

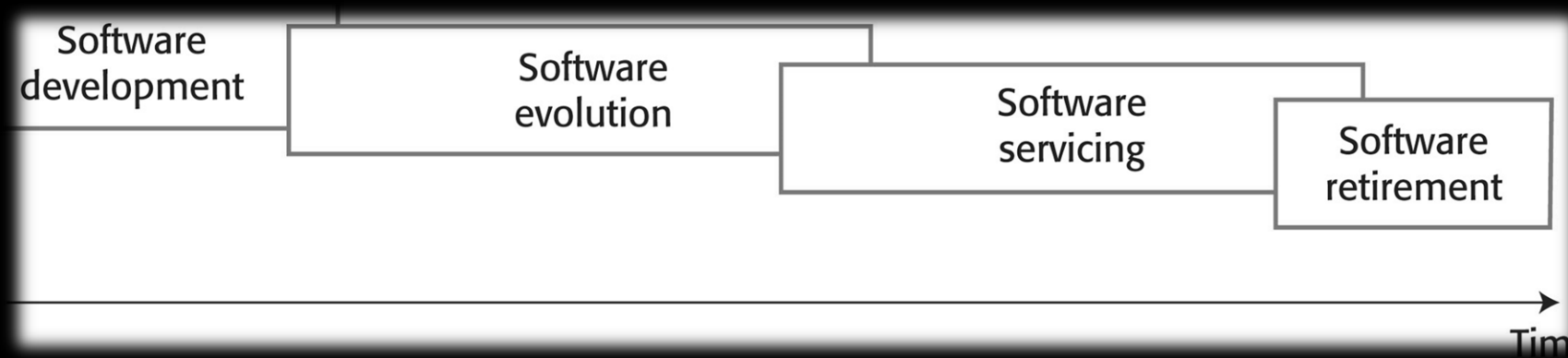


A SPIRAL MODEL OF DEVELOPMENT AND EVOLUTION



EVOLUTION AND SERVICING

- **Evolution:** The stage in a software system's life cycle where it is in operational use and is evolving as new requirements are proposed and implemented
- **Servicing:** At this stage, the software remains useful but the only changes made are those required to keep it operational. No new functionality is added.
- **Phase-out:** The software may still be used but no further changes are made to it.



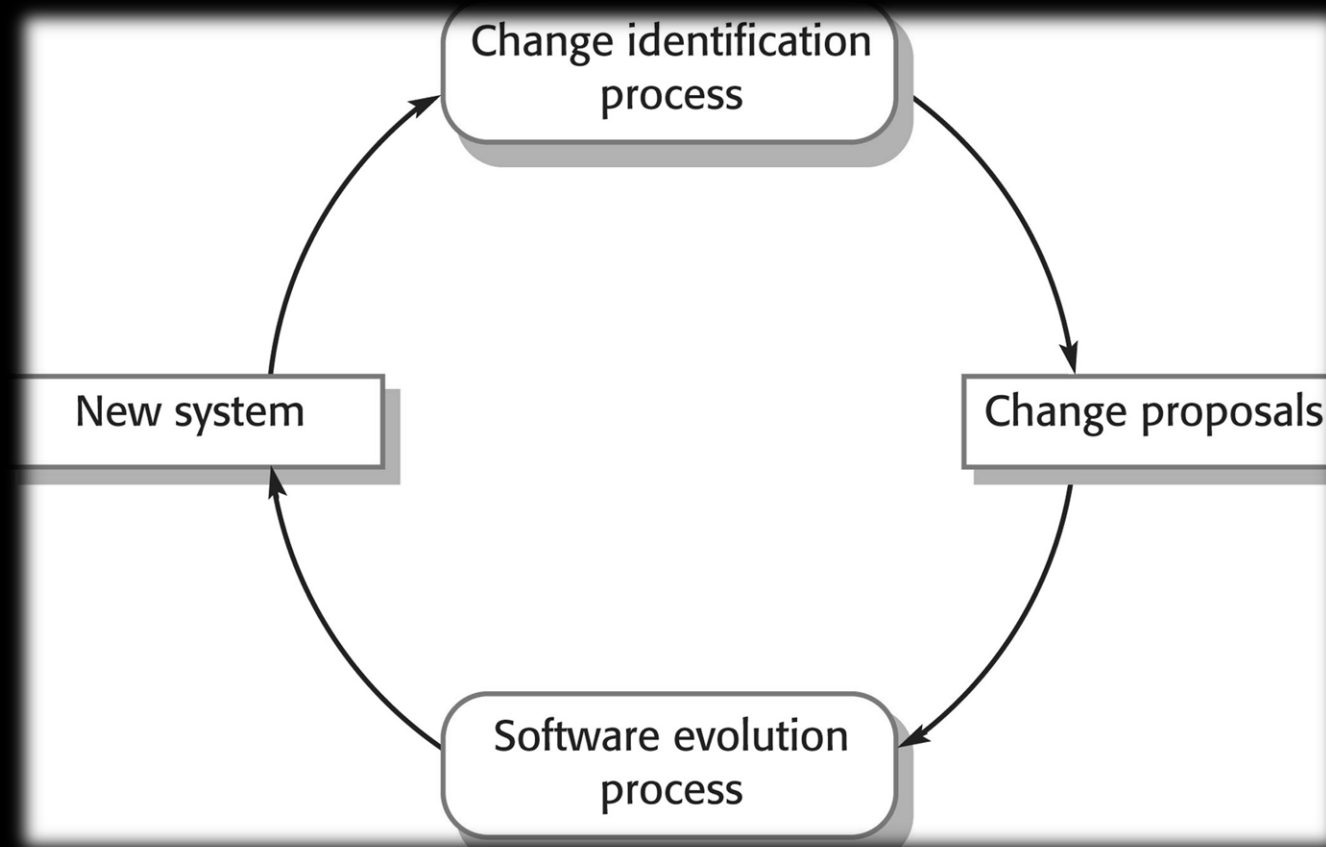


EVOLUTION PROCESSES

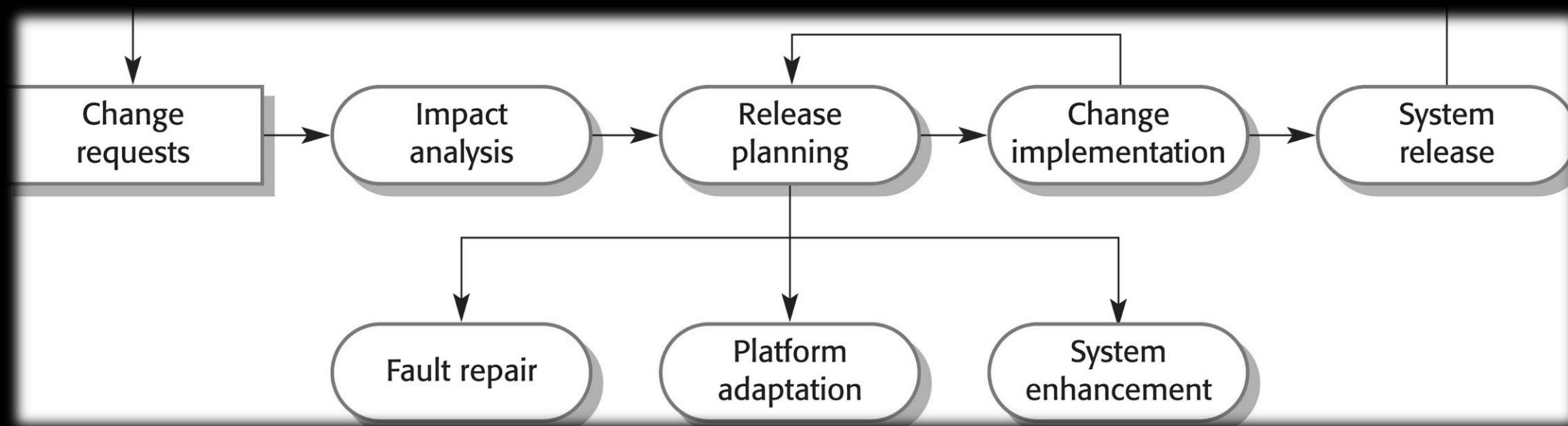
- Software evolution processes depend on
 - The type of software being maintained;
 - The development processes used;
 - The skills and experience of the people involved.
- Proposals for change are the driver for system evolution.
 - Should be linked with components that are affected by the change, thus allowing the cost and impact of the change to be estimated.
- Change identification and evolution continues throughout the system lifetime.



CHANGE IDENTIFICATION AND EVOLUTION PROCESSES

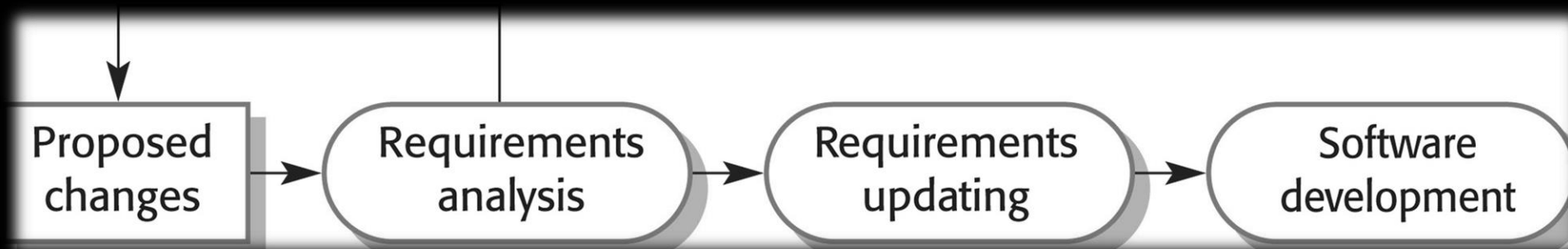


THE SOFTWARE EVOLUTION PROCESS



CHANGE IMPLEMENTATION

- Iteration of the development process where the revisions to the system are designed, implemented and tested.
- The first stage of change implementation may involve program understanding, especially if the original system developers are not responsible for the change implementation.
- During the program understanding phase, you have to understand how the program is structured, how it delivers functionality and how the proposed change might affect the program.



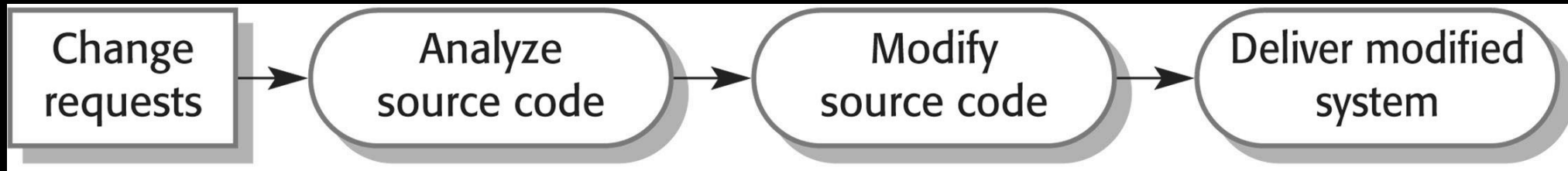


URGENT CHANGE REQUESTS

- Urgent changes may have to be implemented without going through all stages of the software engineering process
 - If a serious system fault has to be repaired to allow normal operation to continue;
 - If changes to the system's environment (e.g. an OS upgrade) have unexpected effects;
 - If there are business changes that require a very rapid response (e.g. the release of a competing product).



THE EMERGENCY REPAIR PROCESS





AGILE METHODS AND EVOLUTION

- Agile methods are based on incremental development so the transition from development to evolution is a seamless one.
 - Evolution is simply a continuation of the development process based on frequent system releases.
- Automated regression testing is particularly valuable when changes are made to a system.
- Changes may be expressed as additional user stories.





HANDOVER PROBLEMS

- Where the development team have used an agile approach but the evolution team is unfamiliar with agile methods and prefer a plan-based approach.
 - The evolution team may expect detailed documentation to support evolution and this is not produced in agile processes.
- Where a plan-based approach has been used for development but the evolution team prefer to use agile methods.
 - The evolution team may have to start from scratch developing automated tests and the code in the system may not have been refactored and simplified as is expected in agile development.

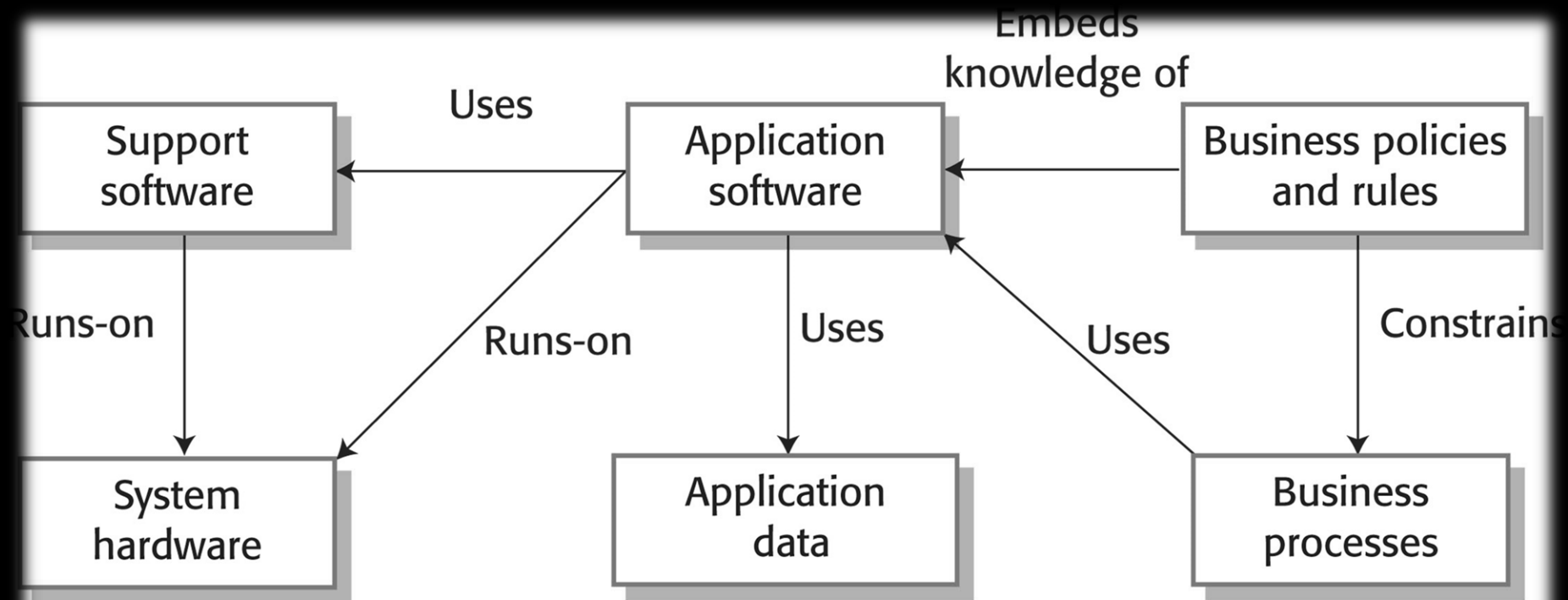


LEGACY SYSTEMS

- »» Legacy systems are older systems that rely on languages and technology that are no longer used for new systems development.
- »» Legacy software may be dependent on older hardware, such as mainframe computers and may have associated legacy processes and procedures.
- »» Legacy systems are not just software systems but are broader socio-technical systems that include hardware, software, libraries and other supporting software and business processes.

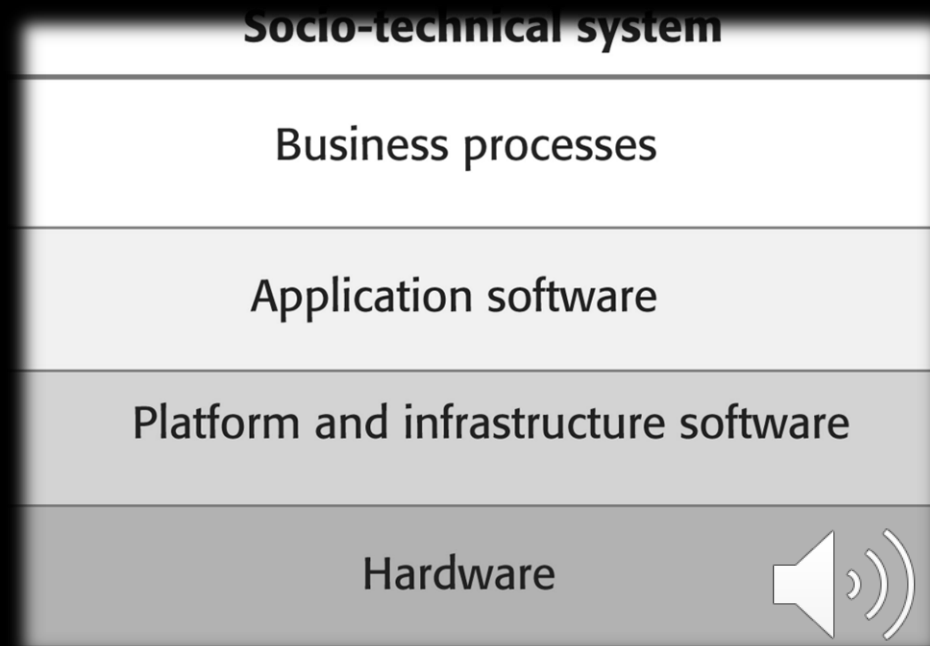


THE ELEMENTS OF A LEGACY SYSTEM



LEGACY SYSTEM COMPONENTS

- 1. Hardware:** Legacy systems often depend on outdated hardware.
- 2. Support Software:** Relies on potentially obsolete or unsupported support software.
- 3. Application Software:** Typically comprises multiple programs delivering business services.
- 4. Application Data:** Processed by the application system, often inconsistent or duplicated across BDs.
- 5. Business Processes:** Aim to fulfill business objectives, possibly restricted by legacy systems.
- 6. Business Policies and Rules:** Define business operations and may be intertwined with legacy system usage.





LEGACY SYSTEM REPLACEMENT

- »» Legacy system replacement is risky and expensive so businesses continue to use these systems
- »» System replacement is risky for a number of reasons
 - Lack of complete system specification
 - Tight integration of system and business processes
 - Undocumented business rules embedded in the legacy system
 - New software development may be late and/or over budget





LEGACY SYSTEM CHANGE

- »» Legacy systems are expensive to change for a number of reasons:
- No consistent programming style
 - Use of obsolete programming languages with few people available with these language skills
 - Inadequate system documentation
 - System structure degradation
 - Program optimizations may make them hard to understand
 - Data errors, duplication and inconsistency





LEGACY SYSTEM MANAGEMENT

- Organisations that rely on legacy systems must choose a strategy for evolving these systems
 - Scrap the system completely and modify business processes so that it is no longer required;
 - Continue maintaining the system;
 - Transform the system by re-engineering to improve its maintainability;
 - Replace the system with a new system.
- The strategy chosen should depend on the system quality and its business value.



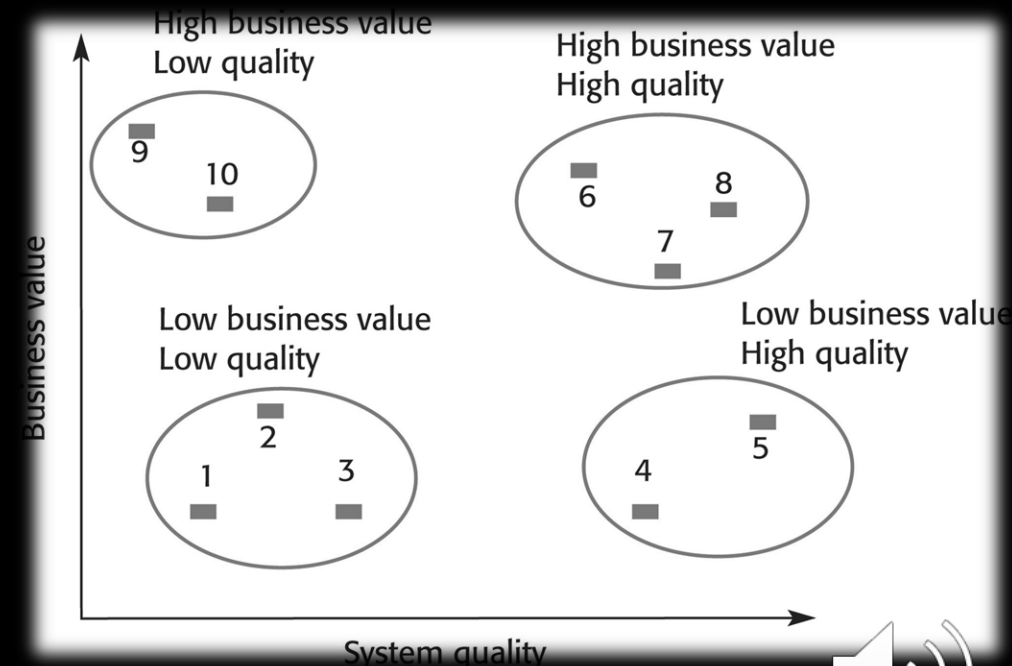
LEGACY SYSTEM CATEGORIES

» Low quality, low business value: These systems should be scrapped.

» **Low-quality, high-business value:**
These make an important business contribution but are expensive to maintain. Should be re-engineered or replaced if a suitable system is available.

» **High-quality, low-business value:** Replace with COTS, scrap completely or maintain.

» **High-quality, high business value:** Continue in operation using normal system maintenance.





BUSINESS VALUE ASSESSMENT

- Assessment should take different viewpoints into account
 - System end-users;
 - Business customers;
 - Line managers;
 - IT managers;
 - Senior managers.
- Interview different stakeholders and collate results.





ISSUES IN BUSINESS VALUE ASSESSMENT

- **The use of the system:** If systems are only used occasionally or by a small number of people, they may have a low business value.
- **The business processes that are supported:** A system may have a low business value if it forces the use of inefficient business processes.
- **System dependability:** If a system is not dependable and the problems directly affect business customers, the system has a low business value.
- **The system outputs:** If the business depends on system outputs, then the system has a high business value.





SYSTEM QUALITY ASSESSMENT

- »» **Business process assessment:** How well does the business process support the current goals of the business?
- »» **Environment assessment:** How effective is the system's environment and how expensive is it to maintain?
- »» **Application assessment:** What is the quality of the application software system?





SYSTEM MEASUREMENT

- You may collect quantitative data to make an assessment of the quality of the application system
 - The number of system change requests; The higher this accumulated value, the lower the quality of the system.
 - The number of different user interfaces used by the system; The more interfaces, the more likely it is that there will be inconsistencies and redundancies in these interfaces.
 - The volume of data used by the system. As the volume of data (number of files, size of database, etc.) processed by the system increases, so too do the inconsistencies and errors in that data.
 - Cleaning up old data is a very expensive and time-consuming process





SOFTWARE MAINTENANCE

- »» Modifying a program after it has been put into use.
- »» The term is mostly used for changing custom software. Generic software products are said to evolve to create new versions.
- »» Maintenance does not normally involve major changes to the system's architecture.
- »» Changes are implemented by modifying existing components and adding new components to the system.





TYPES OF MAINTENANCE

»» Fault repairs

- Changing a system to fix bugs/vulnerabilities and correct deficiencies in the way meets its requirements.

»» Environmental adaptation

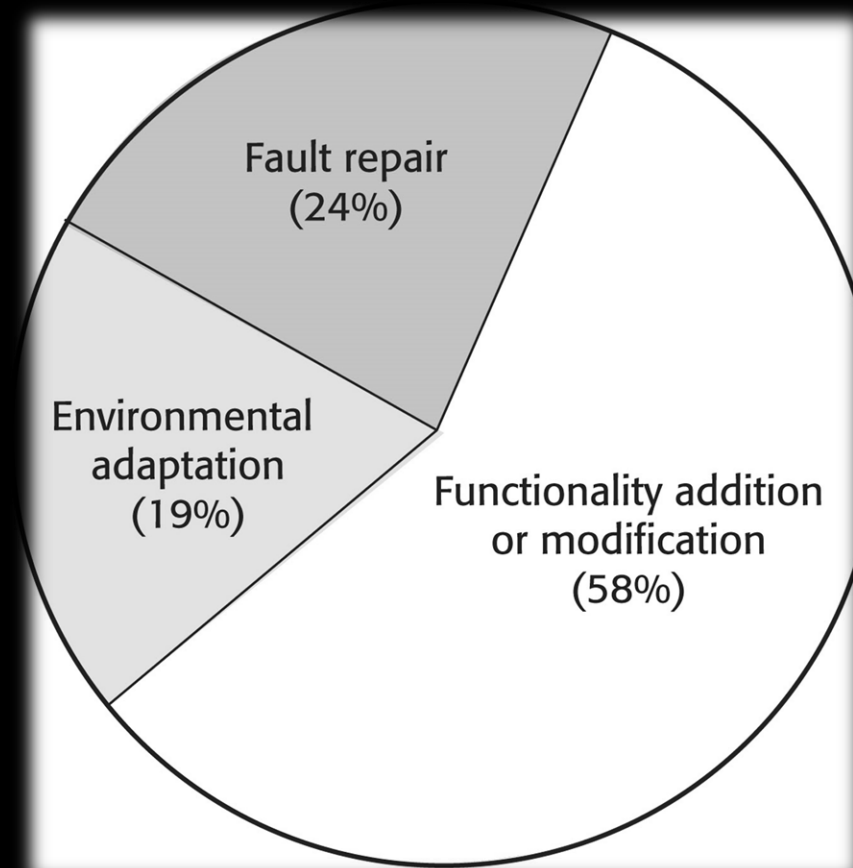
- Maintenance to adapt software to a different operating environment
- Changing a system so that it operates in a different environment (computer, OS, etc.) from its initial implementation.

»» Functionality addition and modification

- Modifying the system to satisfy new requirements.



MAINTENANCE EFFORT DISTRIBUTION





MAINTENANCE COSTS

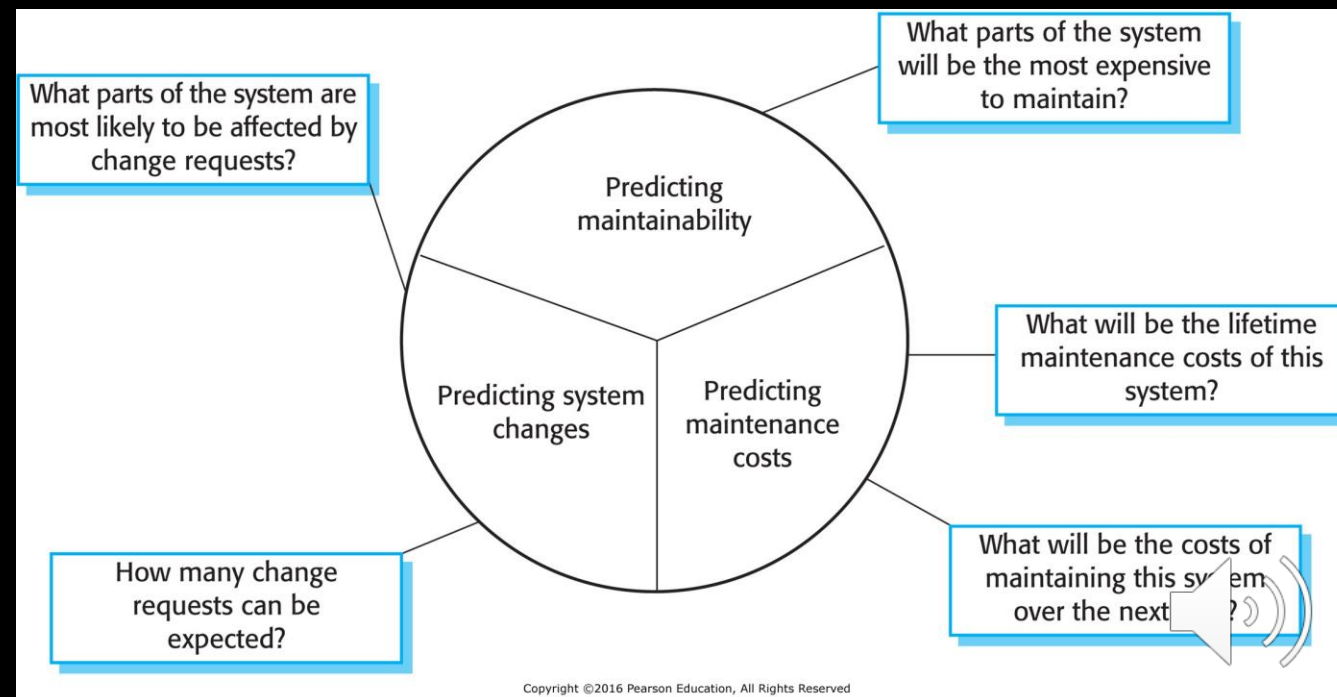
- Usually greater than development costs (2* to 100* depending on the application).
- Affected by both technical and non-technical factors.
- Increases as software is maintained. Maintenance corrupts the software structure so makes further maintenance more difficult.
- Aging software can have high support costs (e.g. old languages, compilers etc.).
- It is usually more expensive to add new features to a system during maintenance than it is to add the same features during development



MAINTENANCE PREDICTION

» Maintenance prediction is concerned with assessing which parts of the system may cause problems and have high maintenance costs

- Change acceptance depends on maintainability of components affected by the change;
- Implementing changes degrades the system and reduces its maintainability;
- Maintenance costs depend on the number of changes and costs of change depend on maintainability.





CHANGE PREDICTION

- »» Predicting the number of changes requires and understanding of the relationships between a system and its environment.
- »» Tightly coupled systems require changes whenever the environment is changed.
- »» Factors influencing this relationship are
 - Number and complexity of system interfaces;
 - Number of inherently volatile system requirements;
 - The business processes where the system is used.





SOFTWARE REENGINEERING

- Restructuring or rewriting part or all of a legacy system without changing its functionality.
- Applicable where some but not all sub-systems of a larger system require frequent maintenance.
- Reengineering involves adding effort to make them easier to maintain. The system may be re-structured and re-documented.





ADVANTAGES OF REENGINEERING

»» Reduced risk

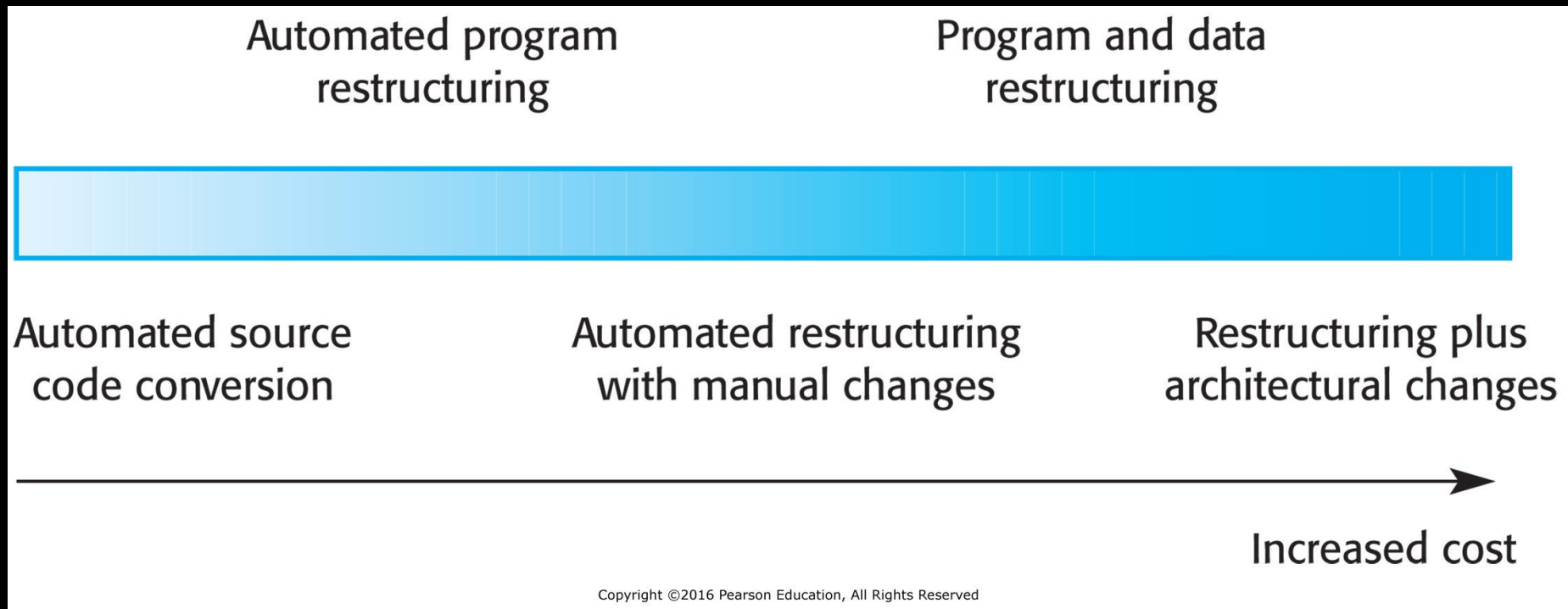
- There is a high risk in new software development. There may be development problems, staffing problems and specification problems.

»» Reduced cost

- The cost of re-engineering is often significantly less than the costs of developing new software.



REENGINEERING APPROACHES





REFACTORING

- »» Refactoring is the process of making improvements to a program to slow down degradation through change.
- »» You can think of refactoring as 'preventative maintenance' that reduces the problems of future change.
- »» Refactoring involves modifying a program to improve its structure, reduce its complexity or make it easier to understand.
- »» When you refactor a program, you should not add functionality but rather concentrate on program improvement.





REFACTORING AND REENGINEERING

- »» Re-engineering takes place after a system has been maintained for some time and maintenance costs are increasing. You use automated tools to process and re-engineer a legacy system to create a new system that is more maintainable.
- »» Refactoring is a continuous process of improvement throughout the development and evolution process. It is intended to avoid the structure and code degradation that increases the costs and difficulties of maintaining a system.





‘BAD SMELLS’ IN PROGRAM CODE

- **Duplicate Code:** Eliminate repeated code by using a single function or method.
- **Long Methods:** Break down lengthy methods into shorter ones.
- **Switch Statements:** Replace repeated switch statements with polymorphism in object-oriented languages.
- **Data Clumping:** Encapsulate recurring data groups into objects.
- **Speculative Generality:** Remove unnecessary generality intended for future use.



